

实验 4 - 自由发挥实验

实验目标：

用 Rust 实现一个功能完整的项目。选题不限，鼓励挑战有难度、有创意的方向。

核心任务：

选题确认：自行选择一个项目方向，在 **GitHub** 上创建仓库并开始开发。

开发实现：使用 **Rust** 编写项目代码，确保功能完整可用。

提交代码：将代码提交至 **GitHub** 仓库，并提交实验报告。

实验 4 评分体系

本方案总分为 100 分，划分为五个核心评分维度。各维度分值分配如下：

评分维度	满分	量化方式
1. 代码规模与复杂度	20	自动统计：有效代码行数、圈复杂度
2. 测试与质量保障	25	自动统计：测试覆盖率、测试用例数、静态分析结果
3. 工程规范	25	自动统计：CI 通过率、Commit规范率、文档覆盖率
4. 项目验收	20	项目完整性与创新性 + 学生自定义验收测试
5. 文档和实验报告	10	结构化评分表：基于检查清单的客观评定
合计	100	

1. 代码规模与复杂度（20 分）

1.1 代码中有效代码行数（12 分）

使用 tokei 工具统计 Rust 源代码行数，排除空行、注释和自动生成代码。有效代码行数直接反映学生的实际编码工作量。我们不希望你们水代码，这没什么意义，所以这一项要求并不高，只是鼓励你实现工作量更大的功能，如果你在编写代码之后发现行数不够，请考虑实现工作量更大的功能，而不是水代码。

有效代码行数（LOC）	得分	说明
LOC < 800	0 分	工作量不足
800 ≤ LOC < 1600	4 分	初级水平
1600 ≤ LOC < 3200	8 分	良好水平
LOC ≥ 3200	12 分	满分

1.2 代码圈复杂度（8 分）

使用 rust-code-analysis-cli 工具计算圈复杂度，衡量代码逻辑复杂程度。圈复杂度反映实现功能的技术深度，过低表示代码过于简单，过高则表示代码需要重构。圈复杂度对于代码质量只是参考作用，真正对代码质量负责的还是自己，所以这一项也没有要求很高。

平均圈复杂度（ACC）	得分	说明
$ACC < 2$	4 分	代码过于简单
$2 \leq ACC < 10$	8 分	复杂度合理（满分）
$ACC \geq 10$	4 分	复杂度过高

2. 测试与质量保障（25 分）

2.1 测试覆盖率（10 分）

使用 cargo-tarpaulin 或 cargo-llvm-cov 工具自动计算你写的测试用例的代码覆盖率。请编写足够的测试。

行覆盖率（Line Coverage）	得分	说明
$Coverage < 30\%$	0 分	覆盖不足
$30\% \leq Coverage < 50\%$	3 分	基础覆盖
$50\% \leq Coverage < 70\%$	6 分	良好覆盖
$70\% \leq Coverage < 85\%$	8 分	优秀覆盖
$Coverage \geq 85\%$	10 分	满分

2.2 测试用例数量（5 分）

统计学生编写的测试用例总数，包括单元测试和集成测试。这里指的是用 cargo test 测试的测试用例数量。

测试用例总数（T）	得分
$T < 5$	0 分
$5 \leq T < 10$	2 分
$10 \leq T < 20$	3 分
$20 \leq T < 30$	4 分
$T \geq 30$	5 分

2.3 静态分析通过率（10 分）

使用 Clippy 进行静态代码分析，统计警告和错误数量。零警告是高质量 Rust 代码的基本要求。

Clippy 警告数 (W)	得分	说明
编译错误	0 分	不可接受
$W > 20$	2 分	质量较差
$10 < W \leq 20$	4 分	需改进
$5 < W \leq 10$	6 分	一般
$1 \leq W \leq 5$	8 分	良好
$W = 0$	10 分	满分

3. 工程规范 (25 分)

3.1 版本控制规范 (10 分)

3.1.1 Commit 消息规范率 (5 分)

使用 commit lint 工具自动检测 Conventional Commits 规范遵循率。规范格式为：

```
type(scope): description
```

规范 Commit 占比 (R)	得分
$R < 50\%$	0 分
$50\% \leq R < 70\%$	1 分
$70\% \leq R < 85\%$	2 分
$85\% \leq R < 95\%$	4 分
$R \geq 95\%$	5 分

3.1.2 Commit 粒度合理性 (5 分)

基于 Commit 数量 (C) 和有效代码行数 (LOC) 的比值评估提交粒度。理想比值范围为每 50-200 行代码一次提交。

平均每 Commit 代码行数 (LOC/C)	得分
$LOC/C < 20$ (过于琐碎)	1 分
$20 \leq LOC/C < 50$	3 分
$50 \leq LOC/C \leq 200$	5 分 (满分)
$200 < LOC/C \leq 500$	3 分
$LOC/C > 500$ (粒度过大)	1 分

3.2 GitHub CI 检查结果 (8 分)

基于 CI 配置文件的客观检查项评分，每满足一项得相应分数。

检查项	分值	验证方式
CI cargo build	2 分	GitHub 检查通过
CI cargo test	2 分	GitHub 检查通过
CI cargo clippy	2 分	GitHub 检查通过
CI cargo fmt --check	2 分	GitHub 检查通过

3.3 新增功能文档完整性（7 分）

使用 cargo doc 和自动化脚本统计文档覆盖情况。

检查项	分值	验证方式
函数文档和测试用例文档覆盖率 $\geq 90\%$	3 分	cargo doc 警告统计
存在 CHANGELOG.md 且格式规范	2 分	文件存在性检查
CLI 命令参数说明完整	1 分	grep 检查
docs 目录中包含专门的说明文档	1 分	grep 检查

4. 项目验收（20 分）

由于实验四是自由发挥项目，每个学生的项目各不相同，无法使用统一的预定义测试用例进行验收。因此本维度分为两部分：

4.1 项目完整性与创新性（10 分）

由助教根据项目的选题难度、功能完整度和设计创新性进行评分。

评分项	分值	说明
选题难度	4 分	项目选题是否有一定技术深度，而非过于简单的练手项目
功能完整度	4 分	项目是否功能完整、可正常运行，而非半成品
设计创新性	2 分	项目是否有独到的设计思路或技术亮点

4.2 学生自定义验收测试（10 分）

要求学生自行编写 ≥ 20 个验收测试用例，覆盖自己项目的核心功能、边界条件和错误处理。助教检查测试质量和覆盖场景的合理性。

评分项	分值	说明
测试用例数量与覆盖面	4 分	是否编写了 ≥ 20 个测试用例，覆盖了主要功能场景
测试场景合理性	3 分	测试是否包含正常路径、边界条件和错误处理三类场景
测试质量	3 分	测试用例是否有意义，而非凑数的无效测试

5. 实验报告质量（10 分）

- **报告内容完整性（5 分）**：实验报告应完整阐述实验目的、设计思路、实现过程和结果分析。具体包括：项目选题的背景和动机、实现方案描述（涉及的模块、数据结构、算法流程等）、遇到的难点及解决办法、功能测试结果展示（可以是截图或日志，证明功能正确性）。内容全面、详实且有理有据可得满分。若报告缺失重要章节（如没有说明设计思路或没有任何测试结果），每缺一项扣 1 分；内容有但过于简略含糊则酌情扣减。
- **报告表达与规范（2 分）**：报告结构清晰，层次分明，文字表述流畅准确。需有适当的标题、段落、列表等 Markdown 排版，方便阅读和打印。图表、示例等辅助手段使用得当。语言表达方面，专业术语使用正确，无明显语法错误或拼写错误。若报告逻辑混乱、语言不通顺，扣 1 分；如语言规范、条理清晰则此项满分。
- **代码注释和项目文档（2 分）**：除了实验报告正文，提交的代码中应包含适当的注释和文档说明。关键模块、复杂算法需有注释解释，项目的 README 应包含使用说明，便于使用者了解。如果完全没有任何注释或说明，酌情扣减 1 分。
- **规范提交与完整性（1 分）**：实验要求通过 GitHub 提交代码和报告，查看提交记录是否规范。学生应使用 Git 进行版本管理，合理编写 commit message。同时检查仓库结构是否完整，例如是否包含 Cargo.toml、Cargo.lock 等必要文件，报告文档是否放在指定位置。

实验报告请以 Issue 形式提交至 GitHub 仓库，报告要求如上，可以用 AI 生成实验报告。